ALEXANDRE Ewan, Groupe 1

AP – GSB

IMPORTANT : La majorité de mes captures d'écrans ont disparus suite à un bug de sauvegarde corrompu… Donc une partie du compte rendu ne présente pas de captures d'écrans.

Table des matières

1) Problématique 2
2) Partie web 2
2.1) Etape 1 : Initialisation du dépôt Git 2
2.2) Etape 2 : Planning du projet avec Gantt 2
2.3) Etape 3 : Supabase 3
2.4) Etape 4 : Connexion de Symfony à Supabase 5
2.5) Etape 5 : Authentification Visiteur et Comptable 6
2.6) Etape 6 : Puissance fiscale des véhicules 7
2.7) Etape 7 : Téléversement des justificatifs 8
2.8) Etape 8/9 : Mise en place du site sur XAMPP et Génération PDF 9
2.9) Etape 10 : Réalisation partie comptable. 10
2.10) Etape 11 : Administration - Gestion des utilisateurs et des comptables. 13
3) Partie mobile 14
3.1) Etape 1 : Configuration et initialisation du projet mobile 14
3.2) Etape 2 : Interdiction de saisie directe des quantités 15
3.3) Etape 3 : Enregistrement des autres catégories de frais forfaitisés 16
3.4) Etape 4 : Suppression de frais hors forfait 17
3.5) Etape 5 : Synchronisation avec la base de données distante Supabase 18
4) Conclusion 18

1) Problématique

Nous avons été chargés de reprendre le projet d'un des développeurs du groupe de pilotage de GSB à partir du travail qu'il a déjà effectué et qu'il nous a fourni. Il va donc falloir rajouter des fonctionnalités tel que : la prise en compte de la puissance fiscale d'un véhicule, la possibilité de téléverser des documents, la possibilité de générer des fichiers PDF des remboursements de frais et l'ajout d'une partie comptable dans l'application. Pour cela nous allons analyser le travail effectué par le développeur, et ajouter/modifier certaines choses à l'aide d'outils comme Gantt, Flutter, Supabase, …

Nous travaillerons d'abord sur la version web avant d'effectuer la version mobile.

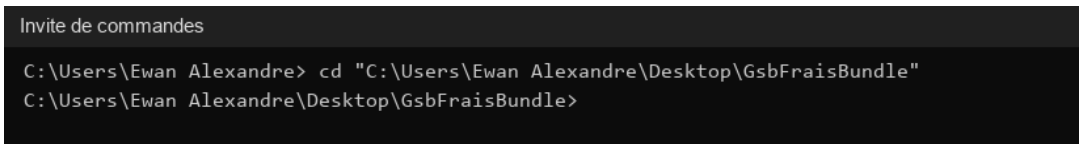
2) Partie web

2.1) Etape 1 : Initialisation du dépôt Git

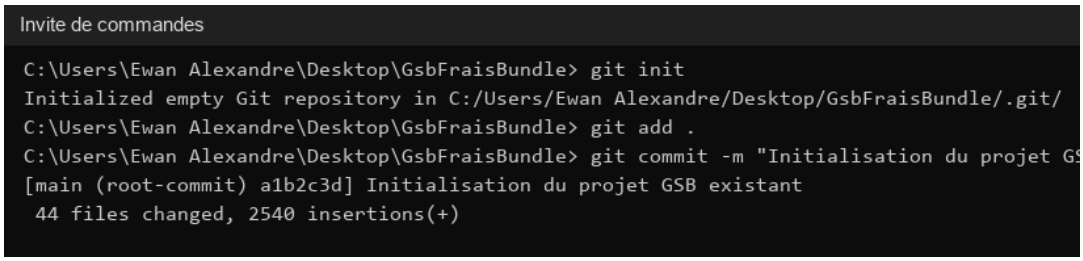
Nous utiliserons Git afin d'enregistrer notre avancement sur le projet, cela nous permettra en cas d'erreur de revenir en arrière pour éviter de rester bloquer sur une version mal programmée.

Pour initialiser notre dépôt git, nous allons utiliser cmd.

Dans cmd on se place sur notre projet grâce à cd comme ici :



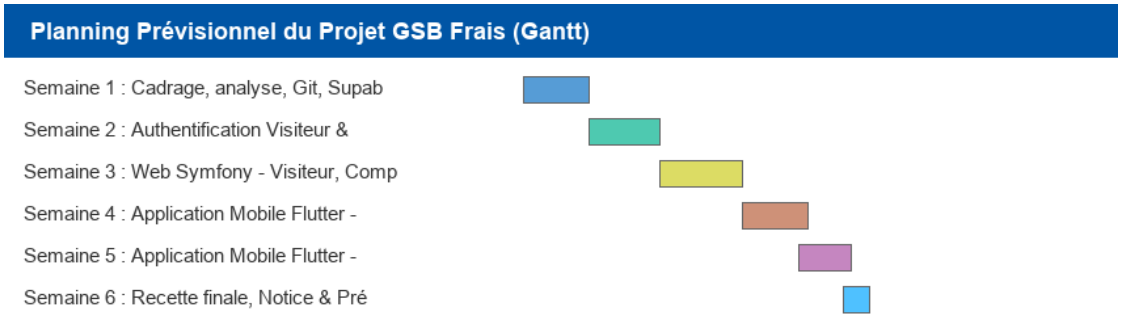
Ensuite on utilise git init pour initialiser le dépôt puis git add . pour ajouter les fichiers au dépôt, et enfin git commit -m ''libellé'' pour donner un nom à la version du projet enregistré :



2.2) Etape 2 : Planning du projet avec Gantt

Nous allons établir un planning prévisionnel des différentes étapes du projet sur le temps qui nous est donné. Il sera aussi disponible sur le cahier des charges, de plus j'ajouterai la vraie version du Gantt si le prévisionnel n'est pas correct.

Voici comment le projet devrait se passer (généré par IA car j'ai perdu la capture originale, les infos sont bonnes par contre) :



2.3) Etape 3 : Supabase

Pour notre base de données, nous utilisons la plateforme Supabase (PostgreSQL).

Nous nous connectons sur supabase.com, nous créons un nouveau projet nommé « GSB », puis nous nous rendons dans l'onglet SQL Editor afin d'exécuter les requêtes de création de nos tables

Voici les différentes requêtes SQL saisies pour construire la base de données :

Table etat

On crée d'abord une table etat, on y ajoute un id char(2) pour représenter l'état d'une fiche durant son cycle de vie. On retrouve cette information dans le fichier créé par le développeur : PdoGsb.php.
On ajoute ensuite aussi un libelle avec NOT NULL à la fin pour forcer l'utilisateur à remplir la variable.
On ajoute ensuite les différents états avec un INSERT INTO … VALUES puis les états données dans le fichier précédemment mentionné.

Table fraisforfait

On crée ensuite une table fraisforfait, elle permet de référencer les 4 forfaits de base et leur tarif unitaire par défaut. On sait comment la construire elle aussi grâce au fichier PdoGsb.php .
On utilise un DECIMAL(5,2) pour le montant afin de ne pas dépasser un forfait à 5 chiffres et à plus de 2 chiffres après la virgule. Puis on ajoute les différents forfaits ainsi que leur « id » donné par le fichier précédemment mentionné.

Table vehicule

On crée ensuite une table vehicule, elle permet de gérer le barème des frais kilométriques selon le véhicule du visiteur. Cette table n'était pas dans le projet de base du développeur, on la crée grâce aux consignes du sujet (le tableau avec les motorisations Diesel/Essence et la puissance fiscale). On utilise un SERIAL pour que l'id s'incrémente tout seul automatiquement. On utilise un DECIMAL(4,2) pour le montant_km afin de stocker les tarifs au kilomètre avec deux chiffres après la virgule (ex: 0.52 ou 0.67). Puis on insère les différents barèmes avec un INSERT INTO.

Table visiteur

On crée la table visiteur, elle permet d'enregistrer les comptes et les coordonnées des visiteurs médicaux. On retrouve ses informations dans le fichier PdoGsb.php. On utilise un char(4) pour l'id car les identifiants des visiteurs font 4 caractères (ex: a17). On met UNIQUE sur le login pour éviter que deux personnes aient le même identifiant. On rajoute aussi la colonne id_vehicule avec REFERENCES vehicule(id) pour relier chaque visiteur à son véhicule pour ses frais. On ajoute ensuite des visiteurs de test avec un INSERT INTO.

Table comptable

On crée la table comptable, elle permet aux comptables de se connecter à leur propre espace pour vérifier et valider les fiches de frais. Cette table n'était pas dans le projet de base, on l'ajoute car elle est demandée dans le sujet (la partie comptable). On utilise un SERIAL pour l'id et un UNIQUE sur le login pour éviter les doublons. On insère ensuite un compte comptable de test avec un INSERT INTO.

Table fichefrais

On crée la table fichefrais, elle représente la fiche mensuelle récapitulative d'un visiteur. On retrouve ses informations dans le fichier PdoGsb.php. On met une clé primaire sur (idvisiteur, mois) pour qu'un visiteur ne puisse avoir qu'une seule fiche par mois. On utilise des DEFAULT comme DEFAULT 'CR' pour que la fiche démarre automatiquement à l'état créée, et DEFAULT 0 pour le nombre de justificatifs et le montant validé. On met ON DELETE CASCADE pour supprimer les fiches si un visiteur est supprimé.

Table lignefraisforfait

On crée la table lignefraisforfait, elle permet de stocker la quantité saisie par le visiteur pour chaque forfait (le nombre de repas, de nuitées, de kilomètres, etc.). On retrouve sa structure dans le fichier PdoGsb.php. On met une clé primaire sur (idvisiteur, mois, idfraisforfait) pour avoir une seule ligne par type de forfait pour un mois donné. On met aussi DEFAULT 0 sur la quantité pour qu'elle commence à zéro à la création.

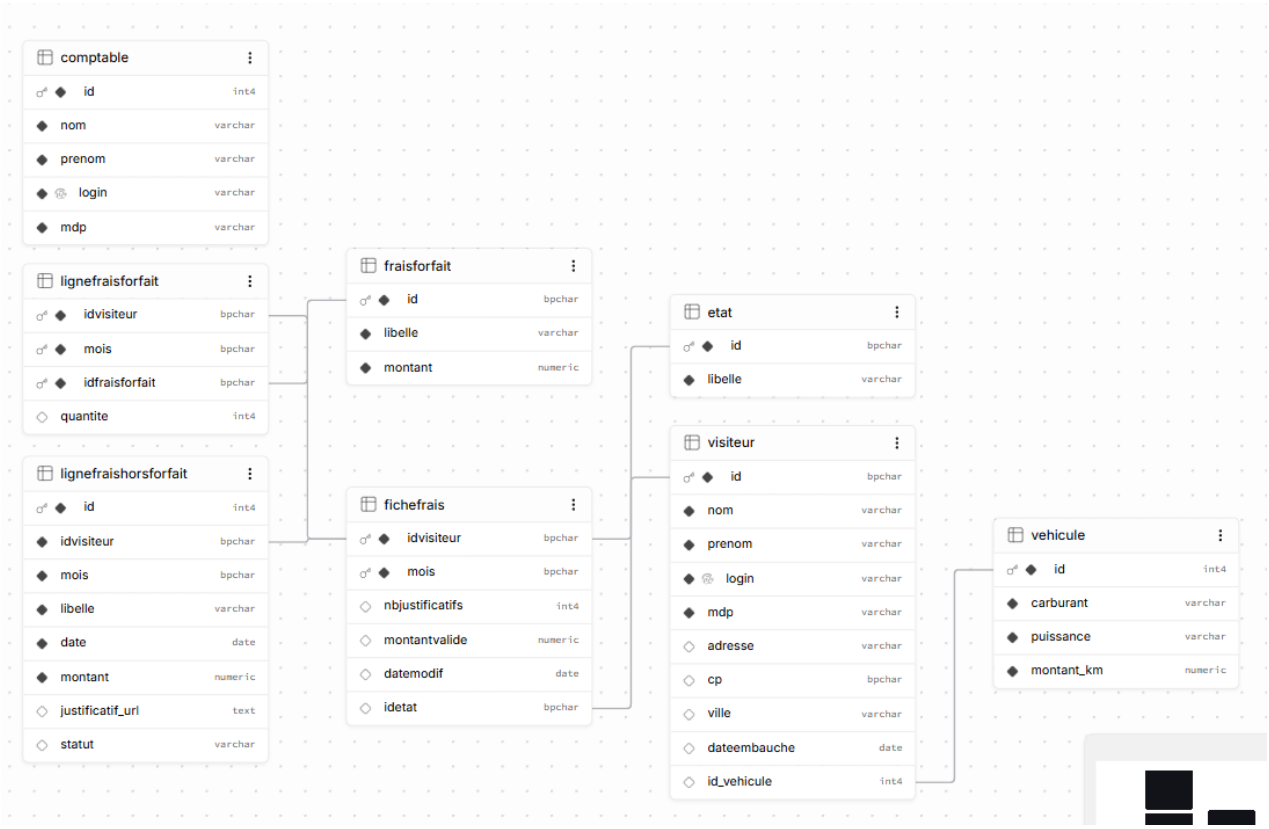
Table lignefraishorsforfait

On crée la table lignefraishorsforfait, elle enregistre les dépenses hors forfait saisies par le visiteur. On retrouve sa structure de base dans le fichier PdoGsb.php. On rajoute la colonne

justificatif_url pour stocker le lien de la facture que le visiteur va téléverser (demandé dans le sujet). On ajoute aussi la colonne statut avec DEFAULT 'En attente' pour permettre au comptable de valider, refuser ou reporter la dépense.

Après ça on peut visualiser la base de données grâce au Schema Visualizer présent dans l'onglet Database. Pour stocker les factures (PDF / images) envoyées par les visiteurs, nous allons dans l'onglet Storage de Supabase, nous créons un nouveau bucket public nommé « justificatifs ».

Résultat :



2.4) Etape 4 : Connexion de Symfony à Supabase

Afin de relier notre application web Symfony à notre base de données Supabase, nous modifions le fichier services.yml présent dans le dossier config

Nous remplaçons les anciens paramètres de connexion MySQL par les paramètres PostgreSQL fournis par Supabase (l'hôte db.vwjbahcpsxetfsmxmqur.supabase.co, le port 5432, la base postgres, l'utilisateur postgres et notre mot de passe).

Ils sont trouvables directement dans l'onglet du bouton Connect de la base de données.

Dans le fichier PdoGsb.php, nous adaptons également le constructeur PDO pour PostgreSQL.

2.5) Etape 5 : Authentification Visiteur et Comptable

Ici on ajoute une fonction get pour la partie comptable du projet :

```
public function getInfosComptable($login, $mdp){
    $req = "select comptable.id as id, comptable.nom as nom, comptable.prenom as prenom from comptable
    where comptable.login=:log and comptable.mdp=:md";
    $stmt = PdoGsb::$monPdo->prepare($req);
    $stmt->bindParam(':log', $login);
    $stmt->bindParam(':md', $mdp);
    $stmt->execute();
    $ligne = $stmt->fetch();
    return $ligne;
}
```

Dans HomeController.php on modifie le if/else pour que la page principale puisse changer selon si on est comptable ou non :

```
$visiteur = $pdo->getInfosVisiteur($login,$mdp);
if(!is_array($visiteur)){
    // Si ce n'est pas un visiteur, on regarde si c'est un comptable
    $comptable = $pdo->getInfosComptable($login,$mdp);
    if(!is_array($comptable)){
        return $this->render('PgGsbFraisBundle:Home:connexion.html.twig',array(
            'message'=>'Erreur de login ou de mot de passe '));
    }
    else{
        $session->set('id',$comptable['id']);
        $session->set('nom',$comptable['nom']);
        $session->set('prenom',$comptable['prenom']);
        $session->set('role','comptable');
        return $this->render('PgGsbFraisBundle::accueil.html.twig');
    }
}
else{
    $session->set('id',$visiteur['id']);
    $session->set('nom',$visiteur['nom']);
    $session->set('prenom',$visiteur['prenom']);
    $session->set('role','visiteur');
    return $this->render('PgGsbFraisBundle::accueil.html.twig');
}
```

Puis dans accueil.html.twig on configure le frontend pour changer d'affichage selon le rôle de l'utilisateur :

```
{% extends "PgGsbFraisBundle::layout.html.twig" %}
{% block menu %}
<div id="menuGauche">
     <div id="infosUtil">
         <h3>
            {% if (app.session.get('nom') is defined) %}
               {% if (app.session.get('role') == 'comptable') %}
                  Comptable : {{ app.session.get('prenom') }} {{ app.session.get('nom') }}<br>
               {% else %}
                  Visiteur : {{ app.session.get('prenom') }} {{ app.session.get('nom') }}<br>
               {% endif %}
            {% endif %}
        </h3>
     </div>
        <ul id="menuList">
          {% if (app.session.get('role') == 'comptable') %}
            <li class="smenu">
               <a href="#" title="Valider les fiches de frais">Valider fiches de frais</a>
            </li>
            <li class="smenu">
               <a href="#" title="Suivi paiement fiches de frais">Suivi paiement fiches</a>
            </li>
          {% else %}
            <li class="smenu">
               <a href="{{ path('pg_gsb_frais_saisirfrais') }}" title="Saisie fiche de frais">Saisie fiche de frais</a>
            </li>
            <li class="smenu">
               <a href="{{ path('pg_gsb_frais_listefrais') }}" title="Consultation de mes fiches de frais">Mes fiches de frais</a>
            </li>
          {% endif %}
            <li class="smenu">
               <a href="{{ path('pg_gsb_frais_deconnexion') }}" title="Se déconnecter">Déconnexion</a>
            </li>
         </ul>
     </div>
{% endblock %}
```

2.6) Etape 6 : Puissance fiscale des véhicules

Nous allons désormais modifier le tarif fixe pour qu'il puisse prendre en compte le type de véhicule des visiteurs.

Nous allons donc effectuer une jointure entre la table visiteur et vehicule en utilisant un INNER JOIN. Cela se fait dans le PdoGsb.php, on y ajoute ceci :

```
/**
 * Retourne le véhicule et le barème kilométrique d'un visiteur
 *
 * @param $idVisiteur
 * @return tableau associatif contenant carburant, puissance, montant_km
 */
    public function getVehiculeVisiteur($idVisiteur){
        $req = "select vehicule.carburant, vehicule.puissance, vehicule.montant_km
                from visiteur inner join vehicule on visiteur.id_vehicule = vehicule.id
                where visiteur.id = :idVisiteur";
        $stmt = PdoGsb::$monPdo->prepare($req);
        $stmt->bindParam(':idVisiteur', $idVisiteur);
        $stmt->execute();
        $leVehicule = $stmt->fetch();
        return $leVehicule;
    }
```

Ensuite on se dirige vers SaisirFraisController.php pour que l'on puisse récupérer le véhicule du visiteur pour que l'on puisse par la suite l'afficher dans un tableau :

```php
$unVehicule = $pdo->getVehiculeVisiteur($idVisiteur);
return $this->render('PgGsbFraisBundle:SaisirFrais:saisirtouslesfrais.html.twig',
array('lesfraisforfait'=>$lesFraisForfait,'lesfraishorsforfait'=>$lesFraisHorsForfait,'nummois'=>$numMois,
    'numannee'=>$numAnnee,'leserreursforfait'=> $lesErreursForfaits,'leserreurshorsforfait'=>null, 'unVehicule'=>$unVehicule));
```

Après ça on va sur saisirfraisforfait.html.twig pour ajouter une condition pour afficher automatiquement le type de véhicule et le tarif au kilomètre du visiteur à côté du champ des frais kilométriques :

```twig
<div id="contenu">
      <h2>Renseigner ma fiche de frais du mois {{nummois}}-{{numannee}}</h2>
      <form action="{{path('pg_gsb_frais_saisirfrais')}}" method="POST">
          <div class="corpsForm">
              <fieldset>
                <legend>Eléments forfaitisés</legend>
                    {% for unfrais in lesfraisforfait %}
                        {% set idfrais = unfrais.idfrais %}
                        {% set libelle = unfrais.libelle %}
                        {% set quantite = unfrais.quantite %}
                        <p>
                            <label for="idFrais">
                                {{libelle}}
                                {% if idfrais == 'KM' and unVehicule is defined and unVehicule %}
                                    <small style="color: #0055A5;">({{ unVehicule.carburant }} {{ unVehicule.puissance }} - {{ unVehicule.montant_km }} €/km)</small>
                                {% endif %}
                            </label>
                            <input type="text" id="idFrais" name="lesFrais[{{idfrais}}]" size="10" maxlength="5" value="{{quantite}}">
                        </p>
                    {% endfor %}
              </fieldset>
          </div>
          <div class="piedForm">
          <p>
            <input id="ok" type="submit" value="Valider" size="20" />
            <input id="annuler" type="reset" value="Effacer" size="20" />
          </p>
          </div>
      </form>
</div>
```

2.7) Etape 7 : Téléversement des justificatifs

Dans saisirfraishorsforfait.html.twig on modifie le code pour ajouter un champ de téléversement ainsi qu'un lien cliquable dans le tableau dans une nouvelle colonne de celui-ci :

```twig

    </table>
      <form action="{{path('pg_gsb_frais_validerfraishorsforfait')}}" method="post" enctype="multipart/form-data">
      <div class="corpsForm">

          <fieldset>
            <legend>Nouvel élément hors forfait
            </legend>
            <p>
              <label for="txtDateHF">Date (jj/mm/aaaa): </label>
              <input type="text" id="txtDateHF" name="dateFrais" size="10" maxlength="10" value=""  />
            </p>
            <p>
              <label for="txtLibelleHF">Libellé</label>
              <input type="text" id="txtLibelleHF" name="libelle" size="70" maxlength="256" value="" />
            </p>
            <p>
              <label for="txtMontantHF">Montant : </label>
              <input type="text" id="txtMontantHF" name="montant" size="10" maxlength="10" value="" />
            </p>
            <p>
              <label for="fileJustificatif">Justificatif (PDF, JPG, PNG) : </label>
              <input type="file" id="fileJustificatif" name="justificatif" accept=".pdf,.png,.jpg,.jpeg" />
            </p>
          </fieldset>
      </div>
      <div class="piedForm">
      <p>
        <input id="ajouter" type="submit" value="Ajouter" size="20" />
        <input id="effacer" type="reset" value="Effacer" size="20" />
      </p>
      </div>

      </form>
</div>
```

Dans SaisirFraisController.php on modifie la méthode validerfraishorsforfaitAction() on vérifie le type de fichier, on lui donne un nom unique, on le place dans un de nos dossiers et on transmet le nom du fichier à la méthode d'insertion SQL.

Enfin dans PdoGsb.php on modifie l'une des méthodes pour que ça puisse ajouter le fichier dans la colonne justificatif_url.

2.8) Etape 8/9 : Mise en place du site sur XAMPP et Génération PDF

Pour faire fonctionner le bundle, j'ai dû avoir recours à Gemini pour mettre en place une copie du bundle dans le dossier htdocs, dans cette copie il y a de nouveaux dossiers nécessaires au fonctionnement de Symfony. Des outils ont dû être installés à l'aide de Composer.

Après ça, j'ai aussi dû installer la bibliothèque FPDF dans Composer qui nous aidera grandement.

On a créé un nouveau fichier dans le dossier services : PdfGsb.php dans lequel on a créé une nouvelle classe utilisant l'outil que l'on vient d'installer et on y a ajouté la méthode ajouterFicheRemboursement() déjà intégré au code :

```php
<?php
namespace Pg\GsbFraisBundle\services;

use FPDF;

require_once __DIR__ . '/../Controller/include/fct.inc.php';

class PdfGsb extends FPDF
{
    private $nomMoisFr = [
        '01' => 'Janvier', '02' => 'Février', '03' => 'Mars', '04' => 'Avril',
        '05' => 'Mai', '06' => 'Juin', '07' => 'Juillet', '08' => 'Août',
        '09' => 'Septembre', '10' => 'Octobre', '11' => 'Novembre', '12' => 'Décembre'
    ];

    public function __construct()
    {
        parent::__construct('P', 'mm', 'A4');
        $this->SetAutoPageBreak(true, 15);
    }

    /**
     * Ajoute une page de fiche de remboursement conforme à l'Annexe 1
     */
    public function ajouterFicheRemboursement($visiteur, $mois, $lesFraisForfait, $lesFraisHorsForfait, $vehicule)
    {
        $this->AddPage();

        $numAnnee = substr($mois, 0, 4);
        $numMois = substr($mois, 4, 2);
        $libelleMois = ($this->nomMoisFr[$numMois] ?? $numMois) . ' ' . $numAnnee;

        // 1. Logo GSB
        $logoPath = __DIR__ . '/../../../../web/bundles/gsbfrais/images/logo.jpg';
        if (file_exists($logoPath)) {
            $this->Image($logoPath, 85, 10, 40);
            $this->Ln(28);
        } else {
            $this->SetFont('Arial', 'B', 22);
            $this->SetTextColor(31, 78, 121);
            $this->Cell(0, 15, 'GSB', 0, 1, 'C');
            $this->Ln(5);
        }

        // 2. Cadre Titre "REMBOURSEMENT DE FRAIS ENGAGES"
        $this->SetDrawColor(31, 78, 121); // Bleu GSB
        $this->SetLineWidth(0.6);
        $this->SetFont('Arial', 'B', 13);
        $this->SetTextColor(31, 78, 121);
        $this->Cell(0, 10, 'REMBOURSEMENT DE FRAIS ENGAGES', 1, 1, 'C');
        $this->Ln(6);

        // 3. Cadre Informations Visiteur & Mois
        $this->SetFont('Arial', '', 11);
        $this->SetTextColor(0, 0, 0);

        // Ligne Visiteur
        $this->SetFont('Arial', 'B', 10);
```

Ensuite on a modifié le PdoGsb.php pouur ajouter les requêtes à la base de données Supabase avec deux nouvelles méthodes :

```php
*/
    public function getInfosVisiteurParId($idVisiteur){
        $req = "select visiteur.id, visiteur.nom, visiteur.prenom, visiteur.adresse, visiteur.cp, visiteur.ville, visiteur.id_vehicule
                from visiteur where visiteur.id = :idVisiteur";
        $stmt = PdoGsb::$monPdo->prepare($req);
        $stmt->bindParam(':idVisiteur', $idVisiteur);
        $stmt->execute();
        return $stmt->fetch();
    }

/**
 * Retourne toutes les fiches clôturées ou validées d'un mois donné
 * @param $mois
 * @return array
 */
    public function getLesFichesClotureesMois($mois){
        $req = "select fichefrais.idvisiteur, fichefrais.mois, fichefrais.nbjustificatifs,
                    fichefrais.montantvalide, fichefrais.datemodif, fichefrais.idetat,
                    visiteur.nom, visiteur.prenom
                from fichefrais
                inner join visiteur on fichefrais.idvisiteur = visiteur.id
                where fichefrais.mois = :mois and (fichefrais.idetat = 'CL' or fichefrais.idetat = 'VA')
                order by visiteur.nom, visiteur.prenom";
        $stmt = PdoGsb::$monPdo->prepare($req);
        $stmt->bindParam(':mois', $mois);
        $stmt->execute();
        return $stmt->fetchAll();
    }
}
?>
```

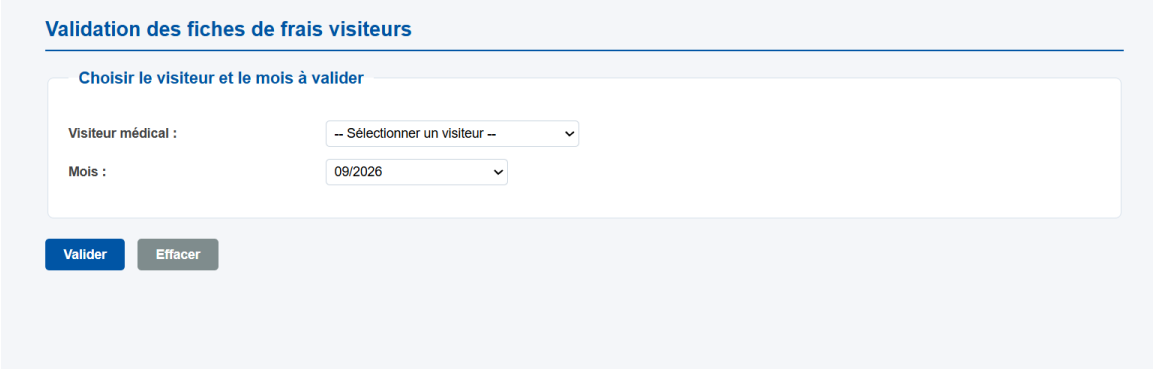
Dans ListeFraisController.php on a ajouté deux méthodes pour la génération des fiches PDF.

Puis pour finir on a modifié le front-end (fichier twig) pour ajouter les boutons correspondants à la fonction de génération des PDFs.

2.9) Etape 10 : Réalisation partie comptable.

Conformément à l'Annexe 2 du cahier des charges, nous développons le module comptable permettant à l'agent comptable de contrôler, modifier et valider les remboursements de frais des visiteurs médicaux.

1. Conception de l'interface utilisateur (UI) :



Nous créons la vue validerfrais.html.twig qui structure l'écran en plusieurs blocs :

- Un bandeau supérieur de sélection avec deux listes déroulantes (Visiteur médical et Mois).

- Un formulaire d'actualisation des quantités forfaitisées.

- Un tableau des frais hors forfait intégrant la visualisation des justificatifs et des boutons d'actions (Refuser et Reporter).

- Un bloc de validation finale avec la saisie du nombre de justificatifs reçus.

2. Développement du contrôleur ValiderFraisController.php :

```php
require_once("include/fct.inc.php");
use Symfony\Bundle\FrameworkBundle\Controller\Controller;
use Symfony\Component\HttpFoundation\Session\Session;
use Symfony\Component\HttpFoundation\Response;

class ValiderFraisController extends Controller
{
    public function indexAction()
    {
        $session = $this->get('request')->getSession();
        if ($session->get('role') !== 'comptable') {
            return $this->render('PgGsbFraisBundle:Home:connexion.html.twig', array(
                'message' => 'Accès réservé aux comptables'
            ));
        }

        $pdo = $this->get('pg_gsb_frais.pdo');
        $request = $this->get('request');

        $lesVisiteurs = $pdo->getTousLesVisiteurs();

        // Liste des 6 derniers mois
        $lesMois = array();
        for ($i = 0; $i < 6; $i++) {
            $timestamp = strtotime("-$i month");
            $cleMois = date('Ym', $timestamp);
            $numAnnee = date('Y', $timestamp);
            $numMois = date('m', $timestamp);
            $lesMois[$cleMois] = array(
                'mois' => $cleMois,
                'numAnnee' => $numAnnee,
                'numMois' => $numMois
            );
        }

        $idVisiteurSelectionne = $request->query->get('lstVisiteur', $request->request->get('lstVisiteur'));
        $moisSelectionne = $request->query->get('lstMois', $request->request->get('lstMois'));
        $message = null;
        $messageType = 'info';

        // 1. Action : Actualiser les frais forfaitisés
        if ($request->getMethod() == 'POST' && $request->request->get('action') === 'actualiserForfait') {
            $lesFrais = $request->request->get('lesFrais');
            if (lesQteFraisValides($lesFrais)) {
                $pdo->majFraisForfait($idVisiteurSelectionne, $moisSelectionne, $lesFrais);
                $message = "Les frais forfaitisés ont été mis à jour avec succès.";
                $messageType = 'success';
            } else {
                $message = "Erreur : les quantités doivent être des valeurs numériques.";
                $messageType = 'error';
            }
        }

        // 2. Action : Refuser un frais hors forfait
        if ($request->getMethod() == 'POST' && $request->request->get('action') === 'refuserHF') {
```

Le contrôleur ValiderFraisController sécurise l'accès en vérifiant le rôle en session (role === 'comptable') et traite les différentes requêtes POST : actualisation des forfaits, refus ou report d'un hors-forfait, et validation globale de la fiche.

3. Traitements métiers dans PdoGsb.php :

```php
public function reporterFraisHorsForfait($idFrais, $idVisiteur, $mois){
    $annee = (int)substr($mois, 0, 4);
    $numMois = (int)substr($mois, 4, 2);
    $numMoisSuivant = $numMois + 1;
    $anneeSuivante = $annee;
    if ($numMoisSuivant > 12) {
        $numMoisSuivant = 1;
        $anneeSuivante++;
    }
    $moisSuivant = sprintf('%04d%02d', $anneeSuivante, $numMoisSuivant);

    if ($this->estPremierFraisMois($idVisiteur, $moisSuivant)) {
        $this->creeNouvellesLignesFrais($idVisiteur, $moisSuivant);
    }

    $req = "update lignefraishorsforfait set mois = :moisSuivant where id = :idFrais";
    $stmt = PdoGsb::$monPdo->prepare($req);
    $stmt->bindParam(':moisSuivant', $moisSuivant);
    $stmt->bindParam(':idFrais', $idFrais);
    $stmt->execute();
}

/*
Valide une fiche de frais : calcule le montant total et passe l'état à 'VA'
@param $idVisiteur
@param $mois
@param $nbJustificatifs
*/
public function validerFicheFrais($idVisiteur, $mois, $nbJustificatifs){
    $lesFraisForfait = $this->getLesFraisForfait($idVisiteur, $mois);
    $vehicule = $this->getVehiculeVisiteur($idVisiteur);
    $totalForfait = 0;
    foreach ($lesFraisForfait as $unFrais) {
        $qte = (float)($unFrais['quantite'] ?? 0);
        $montantUnitaire = (float)($unFrais['montant'] ?? 0);
        if ($unFrais['idfrais'] === 'KM' && !empty($vehicule['montant_km'])) {
            $montantUnitaire = (float)$vehicule['montant_km'];
        }
        $totalForfait += $qte * $montantUnitaire;
    }

    $lesFraisHF = $this->getLesFraisHorsForfait($idVisiteur, $mois);
    $totalHF = 0;
    foreach ($lesFraisHF as $unFraisHF) {
        if (strpos($unFraisHF['libelle'], 'REFUSE : ') !== 0) {
            $totalHF += (float)($unFraisHF['montant'] ?? 0);
        }
    }

    $totalValide = $totalForfait + $totalHF;

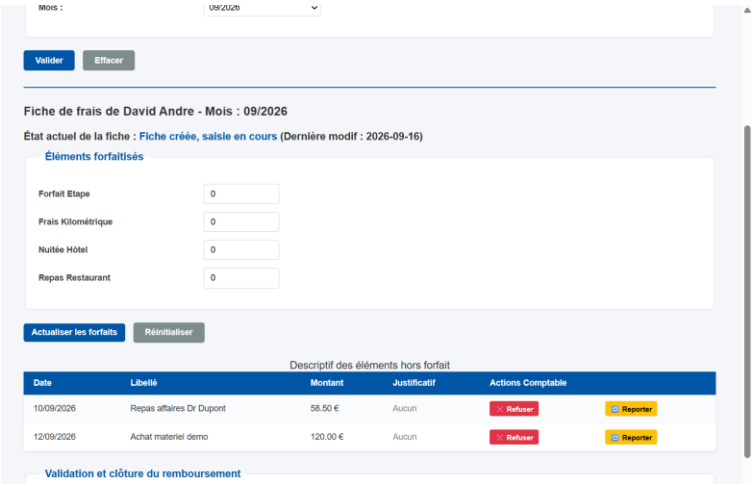
    $req = "update fichefrais set idetat = 'VA', montantvalide = :totalValide, nbjustificatifs = :nbJustificatifs, datemodif = now()
```

refuserFraisHorsForfait() : ajoute le préfixe REFUSE : au libellé du frais sans le supprimer afin de conserver la traçabilité comptable.

reporterFraisHorsForfait() : décale la ligne sur la fiche du mois suivant (mois + 1).

validerFicheFrais() : calcule automatiquement le montant total validé (somme des forfaits calculés au barème et des hors-forfaits acceptés) et passe l'état de la fiche à VA (Validée).

4. Résultat des tests comptables :

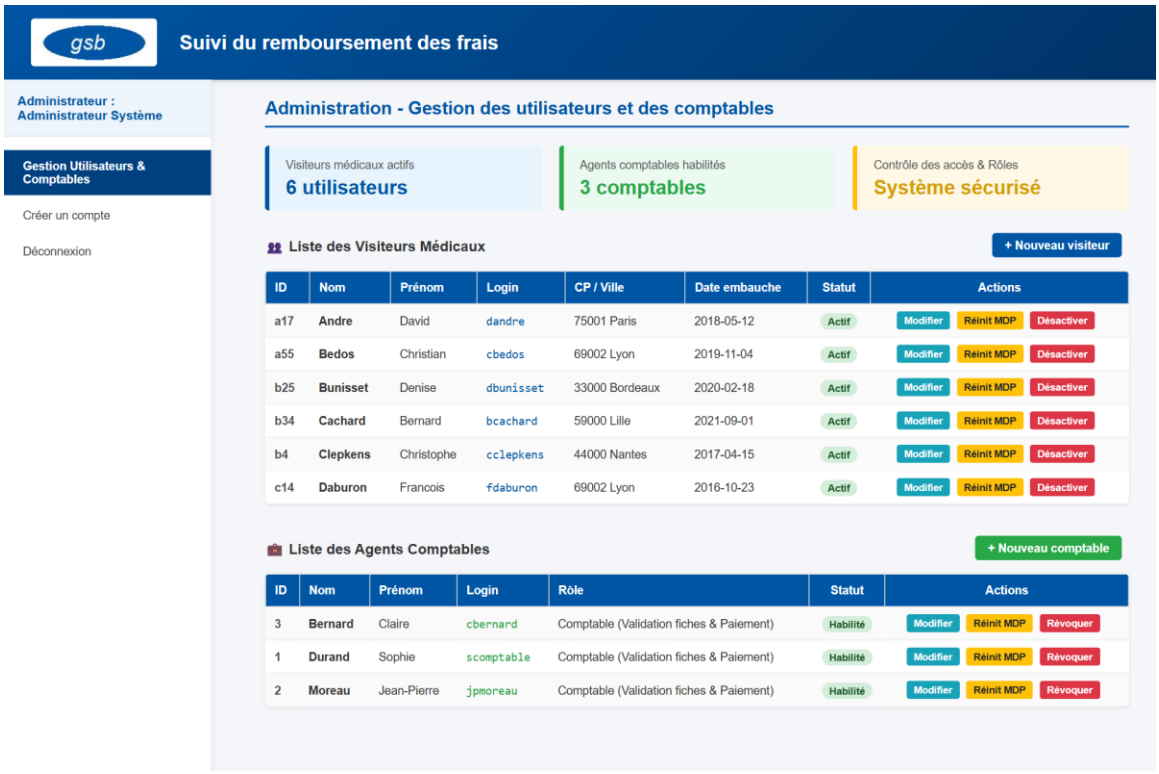


Le comptable peut sélectionner n'importe quel visiteur, ajuster les montants et clôturer la fiche pour déclencher la mise en paiement.

2.10) Etape 11 : Administration - Gestion des utilisateurs et des comptables.

Afin de permettre la gestion globale du système et des comptes par un profil administrateur, nous développons un espace d'administration dédié à la gestion des visiteurs médicaux et des agents comptables.

1. Conception de l'interface utilisateur (UI) :



Nous créons la vue gestionutilisateurs.html.twig qui permet à l'administrateur de superviser et de gérer l'ensemble des utilisateurs :

- Un menu latéral gauche spécifique à l'administrateur permettant d'accéder à la gestion des comptes, à la création d'un utilisateur et à la déconnexion.

- Des cartes récapitulatives affichant le nombre de visiteurs médicaux actifs et d'agents comptables habilités, avec un indicateur de sécurité du système.

- Un tableau récapitulant la liste des visiteurs médicaux (ID, nom, prénom, login, ville, date d'embauche, statut) avec des boutons d'actions pour modifier un profil, réinitialiser le mot de passe ou désactiver le compte.

- Un second tableau listant les agents comptables habilités avec leur rôle et les actions d'administration correspondantes (Modifier, Réinitialiser MDP, Révoquer).

2. Développement du contrôleur AdminController.php :

Nous développons le contrôleur AdminController.php qui sécurise l'accès en vérifiant en session que l'utilisateur possède bien le rôle administrateur. Il interroge la couche de données pour récupérer la liste complète des visiteurs et des comptables, puis transmet ces données à la vue Twig.

3. Traitements métiers dans PdoGsb.php :

getTousLesComptables() : retourne la liste de l'ensemble des agents comptables enregistrés avec leur identifiant, nom, prénom et login.

getInfosAdmin() : vérifie les identifiants de connexion spécifiques à l'administrateur dans la base de données.

getTousLesVisiteurs() : méthode complétée afin de remonter l'ensemble des informations administratives des visiteurs médicaux (identifiant, nom, prénom, login, code postal, ville, date d'embauche).

4. Résultat des tests de l'espace administration :

L'administrateur dispose d'une visibilité complète sur tous les acteurs du laboratoire GSB, ce qui lui permet de gérer efficacement les droits d'accès, d'habiliter les comptables et de maintenir la sécurité des comptes utilisateurs.

3) Partie mobile

3.1) Etape 1 : Configuration et initialisation du projet mobile

L'application mobile a pour objectif de permettre aux visiteurs médicaux de saisir en direct leurs frais (forfaitisés et hors forfait) lors de leurs déplacements professionnels, sans avoir besoin d'attendre d'être devant leur ordinateur.

Dans un premier temps, nous récupérons et initialisons le projet sous notre environnement de développement (Android Studio / VS Code). Nous configurons le SDK et vérifions l'arborescence du projet.

L'application repose sur une architecture claire avec un menu principal ergonomique présentant différentes icônes d'accès rapide : Frais kilométriques, Repas au restaurant, Nuitées d'hôtel, Soirées étape, Saisie des frais hors forfait, Récapitulatif mensuel et Synchronisation distante.

Pour permettre à l'application de fonctionner même hors-ligne (en zone blanche sans réseau 4G/5G), nous mettons en place un système de stockage et de sérialisation locale. Cela permet au visiteur de conserver ses données enregistrées sur son smartphone d'une session à l'autre.

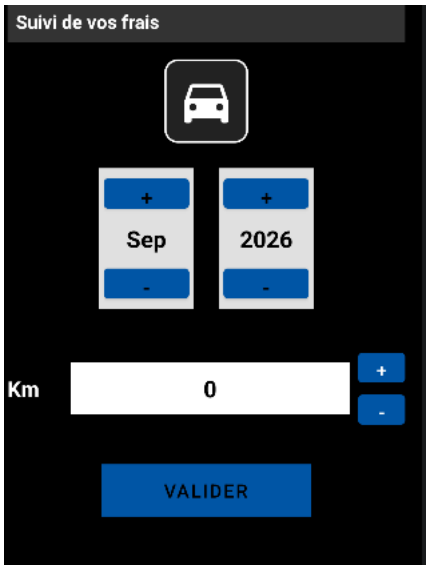


3.2) Etape 2 : Interdiction de saisie directe des quantités

Conformément aux consignes de la Tâche 2 du sujet, nous devons empêcher les visiteurs de saisir directement des chiffres au clavier dans le champ quantité des frais kilométriques afin d'éviter les erreurs de saisie (mauvais caractères, valeurs négatives ou frappes involontaires).

Pour cela, nous modifions le champ de saisie afin de le rendre non modifiable directement (en lecture seule). La quantité ne peut désormais être modifiée qu'en utilisant exclusivement les boutons « + » (incrémentation) et « - » (décrémentation).

Nous ajoutons également une condition de sécurité dans le code afin que la quantité ne puisse jamais être inférieure à zéro.



3.3) Etape 3 : Enregistrement des autres catégories de frais forfaitisés

Dans le projet initial, seule l'activité pour les kilomètres était présente. Nous développons donc les trois autres écrans de saisie pour les catégories de frais forfaitisés manquantes, en respectant la même charte graphique et la même ergonomie :

- Repas restaurant (icône fourchette / couteau) : permet de renseigner le nombre de repas pris au restaurant.

- Nuitée hôtel (icône lit) : permet de renseigner le nombre de nuits d'hôtel.

- Soirée étape (icône tasse / étape) : permet de saisir le nombre de forfaits étape.

Chaque écran comporte le sélecteur de mois/année avec les boutons + et -, l'affichage de la quantité avec ses boutons d'ajustement + et -, ainsi qu'un bouton « Valider » pour sauvegarder localement les informations saisies dans la mémoire du téléphone.

Nous relions ensuite chacun de ces nouveaux écrans aux boutons correspondants du menu principal.



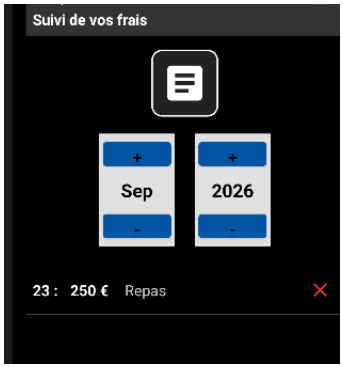
3.4) Etape 4 : Suppression de frais hors forfait

Dans l'activité de consultation du récapitulatif mensuel des frais hors forfait, le visiteur doit pouvoir visualiser l'ensemble de ses dépenses du mois (jour, libellé et montant en euros) et supprimer une ligne en cas d'erreur de saisie.

Pour réaliser cette fonctionnalité (Tâche 4) :

Nous modifions la mise en page de la liste (l'adaptateur de vue / FraisHfAdapter) pour ajouter un bouton rouge avec une croix au bout de chaque ligne de frais hors forfait.

Nous codons l'événement de clic sur ce bouton afin de supprimer l'élément sélectionné de la liste et de mettre à jour immédiatement les données enregistrées dans la mémoire locale du téléphone.



3.5) Etape 5 : Synchronisation avec la base de données distante Supabase

La dernière étape consiste à relier l'application mobile à notre base de données distante Supabase (PostgreSQL) grâce au bouton de synchronisation présent sur le menu principal.

Cette fonctionnalité permet de transférer automatiquement tous les frais saisis hors-ligne sur le téléphone directement vers la base de données centrale de l'entreprise GSB, sans que le visiteur n'ait besoin de tout ressaisir sur le site web.

1. Authentification du visiteur : Pour que la base distante sache à quel visiteur appartiennent les frais reçus, nous demandons la saisie du login et du mot de passe du visiteur médical.

2. Envoi des données vers Supabase : Lors du clic sur le bouton de synchronisation, l'application se connecte à Supabase, vérifie l'existence de la fiche du mois dans la table fichefrais, met à jour les quantités forfaitisées dans lignefraisforfait et insère les lignes dans lignefraishorsforfait.

3. Confirmation : Un message de confirmation s'affiche à l'écran pour indiquer que la synchronisation a été effectuée avec succès.



4) Conclusion

En conclusion, nous avons réussi à faire évoluer l'application GSB en reliant la partie web et la partie mobile à une base de données Supabase. Grâce au module comptable sous Symfony et aux fonctionnalités développées sur l'application mobile (sécurisation des saisies, gestion des forfaits et synchronisation), les visiteurs médicaux peuvent désormais enregistrer leurs frais directement sur leur téléphone et les comptables peuvent les contrôler, les valider et générer les remboursements en PDF.